

Introduction

If you've spent any time on LeetCode's hard-difficulty string problems, you've probably noticed a recurring trap: a problem that looks like a simple linear scan turns hard the moment someone adds the word "update." LeetCode 2213, *Longest Substring of One Repeating Character*, is exactly that trap. The base question — find the longest run of the same character in a string — is something most people can solve in their sleep. What makes it hard is that you have to answer that question again, instantly, after every single character update, potentially tens of thousands of times.

This pattern is worth learning for reasons beyond the LeetCode badge. Any system where data keeps changing but you need a live, correct answer — a collaborative text editor recalculating formatting spans, a streaming analytics dashboard recomputing a rolling metric, a compression tool re-evaluating run-length encoding — leans on the same idea: don't recompute everything, recompute only what changed, and summarize the rest.

Interviewers like this problem because it tests something more specific than "can you code a sliding window." It tests whether you notice that an edit has a *local* blast radius, and whether you know a data structure that turns "local change, global answer" into a cheap operation. That data structure is a segment tree, and once you've built one for this problem, you'll recognize the shape immediately in other problems.

Problem Overview

You're given a string `s`, plus two parallel lists: a list of characters and a list of indices. For each pair `(queryCharacters[i], queryIndices[i])`, you place that character at that index in the string — overwriting whatever was there. After each placement, you need to report the length of the longest contiguous run of identical characters anywhere in the string.

The constraint that makes this interesting: the string and the number of queries can each be up to 100,000. A naive "rescan the whole string after every edit" approach is going to be far too slow.

Example

Start with `s = "babacc"` (as a list: `b, a, b, a, c, c`).

Query 1: place `b` at index 1 → string becomes `b, b, b, a, c, c`. The longest run is `bbb`, length **3**.

Query 2: place `c` at index 3 → string becomes `b, b, b, c, c, c`. Now there are two runs of length 3 (`bbb` and `ccc`). The answer is still **3**.

Query 3: place `b` at index 3 → string becomes `b, b, b, b, c, c`. Now `bbbb` is a run of length **4**.

Final answers: `[3, 3, 4]`.

Notice that every edit changed exactly one character, but the length of the affected run changed by a lot — the second `b` insertion merged what used to be a boundary into a longer run. That's the core difficulty: a single-character change can ripple into a longer run just by connecting two neighboring runs.

Intuition

The instinct most people reach for first is the correct one for the *static* version of this problem: walk the string left to right with a counter. Same character as before, bump the counter; different character, reset it to 1 and remember the best count seen so far. That's an $O(n)$ scan, and it's completely correct — for a string that never changes.

The moment updates enter the picture, an experienced engineer asks a different question: *when I change one character, how much of my previous answer is actually invalidated?* The key realization is that an edit at index `i` can only break or merge runs immediately around `i`. A run 50,000 characters away is untouched. So recomputing the entire answer from scratch every time is throwing away almost all of the previous work.

That leads naturally to a "divide the string into chunks, and let each chunk remember a summary of itself" design. If each chunk knows its own longest run, and — critically — the run touching its left edge and the run touching its right edge, then two neighboring chunks can be combined into a summary of the bigger range just by checking whether the left chunk's right-edge character matches the right chunk's left-edge character. That's a segment tree: a binary tree over the string where each node holds `(max_run, left_run, right_run, character)` for its range, and where updating one leaf only requires recomputing the handful of ancestors on the path back to the root — $O(\log n)$ nodes, not $O(n)$.

Brute Force Solution

Idea: After every update, rescan the entire string with the running-counter approach described above.

Algorithm: 1. Apply the update to the string (list). 2. Walk left to right, tracking the current run length and the best seen so far. 3. Append the best run length to the results.

Advantages: Trivial to write, trivial to verify, no auxiliary data structure.

Disadvantages: Every query costs $O(n)$, regardless of how small the actual change was.

Complexity: With n = string length and q = number of queries, total time is $O(n \cdot q)$. Space is $O(n)$ for the string itself. For $n = q = 10^5$, that's on the order of 10^{10} operations — far too slow.

Optimal Solution

Build a **segment tree** over the string. Each leaf represents a single character; each internal node represents a range and stores:

Field	Meaning
<code>max_run</code>	Longest run of one repeated character anywhere inside this range
<code>left_run</code>	Length of the run starting at the range's left edge
<code>right_run</code>	Length of the run ending at the range's right edge
<code>left_char</code> / <code>right_char</code>	The characters at the two edges (or just infer from the string)

Merging two children (`left` , `right`) into a parent:

- `parent.max_run = max(left.max_run, right.max_run)`
- If `left` 's rightmost character equals `right` 's leftmost character, the runs at that seam combine:
- `parent.max_run = max(parent.max_run, left.right_run + right.left_run)`
- `parent.left_run = left.left_run` , unless `left.left_run == len(left)` (the left child is *entirely* one character), in which case it may extend into the right child: `parent.left_run = left.left_run + (right.left_run if chars match else 0)`
- `parent.right_run` is computed symmetrically from the right side.

Build: Recursively build leaves, then merge bottom-up. $O(n)$.

Update: Change one leaf's character, then walk back up to the root, recomputing each ancestor via merge. Only $O(\log n)$ nodes are touched per update, since the tree has height $\log n$ and each level contributes exactly one node on the path from that leaf.

Query: The answer is always sitting at `root.max_run` — no separate query walk needed.

Python Code

```

class SegmentTree:
    def __init__(self, s):
        self.s = list(s)
        self.n = len(self.s)
        # each node: [max_run, left_run, right_run]
        self.tree = [None] * (4 * self.n)
        self._build(1, 0, self.n - 1)

    def _build(self, node, lo, hi):
        if lo == hi:
            self.tree[node] = (1, 1, 1)
            return
        mid = (lo + hi) // 2
        self._build(2 * node, lo, mid)
        self._build(2 * node + 1, mid + 1, hi)
        self.tree[node] = self._merge(node, lo, mid, hi)

    def _merge(self, node, lo, mid, hi):
        left = self.tree[2 * node]
        right = self.tree[2 * node + 1]
        left_len = mid - lo + 1
        right_len = hi - mid

        max_run = max(left[0], right[0])
        left_run, right_run = left[1], right[1]

        if self.s[mid] == self.s[mid + 1]:
            # seam run: end of left range meets start of right range
            max_run = max(max_run, left[2] + right[1])
            if left[1] == left_len:
                left_run = left_len + right[1]
            if right[2] == right_len:
                right_run = right_len + left[2]

        return (max_run, left_run, right_run)

    def update(self, node, lo, hi, idx, char):
        if lo == hi:
            self.s[idx] = char
            self.tree[node] = (1, 1, 1)
            return
        mid = (lo + hi) // 2
        if idx <= mid:
            self.update(2 * node, lo, mid, idx, char)
        else:
            self.update(2 * node + 1, mid + 1, hi, idx, char)
        self.tree[node] = self._merge(node, lo, mid, hi)

    def longest_repeating(self):
        return self.tree[1][0]

```

```
def longestRepeating(s, queryCharacters, queryIndices):
    tree = SegmentTree(s)
    n = len(s)
    answers = []
    for char, idx in zip(queryCharacters, queryIndices):
        tree.update(1, 0, n - 1, idx, char)
        answers.append(tree.longest_repeating())
    return answers
```

Code Walkthrough

- `SegmentTree.__init__` stores the string as a mutable list (so single-character writes are $O(1)$) and allocates a $4 * n$ -sized array for the tree — the standard safe upper bound for a recursive segment tree.
- `_build` recurses down to single-character leaves, each trivially a run of length 1, then merges pairs of children back up.
- `_merge` is where the logic actually lives. It first takes the plain max of the two children's `max_run`. Then it checks the seam — the last character of the left range and the first character of the right range. If they match, a run can cross the boundary, so it checks `left.right_run + right.left_run` as a candidate for the new max. It also has to correct `left_run / right_run` for the *parent* range: if the left child is entirely one repeated character (its `left_run` fills its whole length), the parent's `left_run` can extend into the right child; same logic mirrored for `right_run`.
- `update` walks down to the target leaf, overwrites the character, then re-merges every ancestor on the way back up — exactly the nodes whose ranges include the changed index.
- `longest_repeating` just reads `tree[1][0]`, the root's `max_run`, since the root's range is the whole string.

Dry Run

String: `b a b a c c` (indices 0–5).

Query 1: place `b` at index 1. Leaf 1 becomes `b`. String is now `b b b a c c`. Walking up, merges detect `bbb` at indices 0–2 (three matching characters in a row), giving `max_run = 3`. Result: **3**.

Query 2: place `c` at index 3. String becomes `b b b c c c`. Two separate runs of 3 (`bbb`, `ccc`), no seam match between index 2 (`b`) and index 3 (`c`). `max_run = 3`. Result: **3**.

Query 3: place `b` at index 3. String becomes `b b b b c c`. Now indices 0–3 are all `b`. The seam between the left half and right half of the tree at that boundary matches, `left.right_run +`

`right.left_run` yields 4. Result: 4.

Final: `[3, 3, 4]` — matches the brute-force answer, but each query only touched $O(\log 6)$ nodes instead of rescanning all 6 characters.

Checkpoint: string `a a a a`, place `z` at index 2. Before reading on — what's the longest run?

It's **2**: `a a` then `z` breaks it, then `a a` again. The most common mistake here is only tracking each half's *best* run and forgetting a run can straddle a boundary between children — which is exactly why every node also stores `left_run` and `right_run`, not just `max_run`.

Complexity Analysis

- **Build:** $O(n)$ — each node is visited once, and merge is $O(1)$ work per node.
- **Update:** $O(\log n)$ per query — the path from a leaf to the root has exactly $\log n$ nodes, and each merge is $O(1)$.
- **Total:** $O(n + k \log n)$ for k queries, versus $O(n \cdot k)$ for the brute force. Space is $O(n)$ for the tree (the $4n$ array) plus $O(n)$ for the string.

This is correct because the only nodes whose stored summary can change after a single-character edit are the ones whose range contains that index — and there are exactly $\log n$ of those, one per tree level.

Alternative Solutions

You could maintain a sorted set of run boundaries (start/end index pairs) and manually merge or split neighboring runs on each edit, tracking the current maximum with a max-heap. This avoids building a tree, but the heap needs **lazy deletion** — old run lengths become stale as runs merge or split, and you can't efficiently remove arbitrary entries from a heap. In practice this ends up being more bookkeeping, not less, than the segment tree, for the same asymptotic complexity.

Edge Cases

- **String of length 1:** builds a single-node tree; every update just rewrites that one leaf. Answer is always 1.
- **Placing the character that's already there:** a no-op update — the leaf gets rewritten to the same value, ancestors re-merge to the same result. Harmless but not free (still $O(\log n)$).

- **Update at index 0 or the last index:** the update walk simply goes all the way to that edge of the tree; no special-casing needed.
- **All characters identical:** `max_run` equals the full string length at every node, and any single edit in the middle splits it into (at most) two smaller runs.
- **Two queries hitting the same index back to back:** each is processed independently; the second update simply overwrites the first.

Common Mistakes

1. **Rescanning the whole string on every query** — technically correct, but $O(n \cdot q)$, which times out on large inputs.
2. **Only storing `max_run` per node and forgetting `left_run` / `right_run`** — this silently drops runs that cross a child boundary, producing a wrong (too small) answer.
3. **Failing to propagate `left_run` / `right_run` correctly when a child is entirely one character** — if you don't check whether `left.left_run == left_len` before extending into the sibling, boundary-crossing runs get miscounted.
4. **Off-by-one errors in the seam comparison** — comparing `s[mid]` and `s[mid+1]` (not `s[lo]` / `s[hi]`) is what actually represents the adjacency between the two child ranges.
5. **Forgetting to re-merge every ancestor after an update** — updating only the leaf and the root, skipping intermediate levels, leaves the tree in an inconsistent state for future queries.

Interview Questions

- How would you extend this to support range updates (assign a character to an entire range at once) instead of point updates?
- Could you support "insert" and "delete" operations instead of overwrite-in-place? What breaks?
- How would you adapt the segment tree if you needed the longest run of *any* two allowed characters, not just one repeated character?
- What's the memory tradeoff of an iterative (bottom-up array) segment tree versus this recursive version?

Similar Problems

- **LeetCode 2213 relatives via segment tree pattern:** LeetCode 307 (Range Sum Query – Mutable) — same "point update, range query, tree of summaries" shape, simpler merge function.
- **LeetCode 715 (Range Module)** — tracks ranges with a different structure but shares the "local edit, global query" theme.

- **LeetCode 128 (Longest Consecutive Sequence)** — related in spirit (longest run), but over a static unsorted array, solved with a hash set instead of a tree.
- **LeetCode 5 (Longest Palindromic Substring)** — different technique, but a good companion for building intuition about "runs" and substring properties.

Key Takeaways

The naive rescan-after-every-edit approach costs $O(n)$ per query because it throws away everything it already knew about the string. The fix isn't a smarter scan — it's a structure that only recomputes what actually changed. A segment tree that stores each range's max run plus its left-edge and right-edge runs lets every update touch just $O(\log n)$ nodes, because a change only affects the ancestors of one leaf. The transferable lesson: whenever an update only affects a small, local region of your data, look for a tree that summarizes chunks and recombines summaries on the way up, instead of rescanning everything from scratch.

Watch the Video

For the full visual walkthrough — including the tree diagrams and the live dry run on `babacc` — check out the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series built for developers who want to actually understand the "why" behind interview problems and core computer science patterns — not just memorize a solution to pass a test case.

Call To Action

If this walkthrough helped the segment tree click, give the video a like on YouTube and subscribe for more deep dives like this one. For write-ups, code, and extra breakdowns delivered straight to your inbox, subscribe to the Substack. Drop a comment with your answer to the checkpoint question, and share this with anyone prepping for interviews.

Quick Review

Substring with only repeated chars — what pattern applies?

Sliding window / linear scan — when you see 'longest run of same element', think group boundaries, not brute-force rescanning.

Time complexity of the one-pass solution?

$O(n)$ — each character visited once, comparing to its predecessor.

Space complexity of the one-pass solution?

$O(1)$ — just running counters for current length and best length.

Trickiest bug in the single-pass approach?

Forgetting to compare the final run's length after the loop ends — the last group never triggers a mismatch to close it.

Naive rescan vs one-pass: what's the rescan trap?

Rescanning from each start index re-walks overlapping ground, turning $O(n)$ into $O(n^2)$ for no new info.

Follow-up: what if you need the substring itself, not just length?

Track start index alongside the running counter, update it whenever a new max length is found.